

GPU 101

How a machine for pictures learned to compute $A + B$

Minjae Gwon

POSTECH

September 10, 2026

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

- Rendering and shaders
- GPGPU and CUDA

3. The execution engine

- Warps and SIMT
- SMs and GPU organization

4. Following A + B

- From tensors to threads
- Scheduling and memory
- Results and completion

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

3. The execution engine

4. Following A + B

What should make more sense after this talk?

- **The GPU you observe**

What are VRAM and all those execution resources?

- **The frameworks you use**

What sits below a PyTorch or Hugging Face operation?

- **The platform NVIDIA built**

How did graphics hardware become a general compute platform?

Pictures → Execution engines → $C = A + B$

One image. Very different ways to paint it.



Parallel work becomes easy to see when the output is a picture.

CPU vs. GPU

One paintball at a time.

Many paintballs together.

► [Watch the demo](#)

NVIDIA NVISION 08 / MythBusters demo · video re-upload: HOSTKEY

1. Motivation

2. From pictures to programs

- Rendering and shaders
 - GPGPU and CUDA
-

3. The execution engine

4. Following A + B

The original deadline: draw the next frame

3D rendering

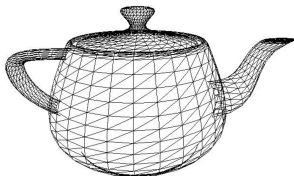


Image credit: Henrik Wann Jensen

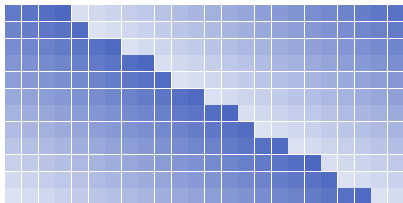
Input: description of a scene:

3D surface geometry (e.g., triangle mesh)
surface materials, lights, camera, etc.

Output: image of the scene

Geometry + materials + lights + camera → an image

Millions of positions. The same kind of calculation.



Different positions, different inputs

1920 × 1080

output pixels per frame

× **60 frames / second**

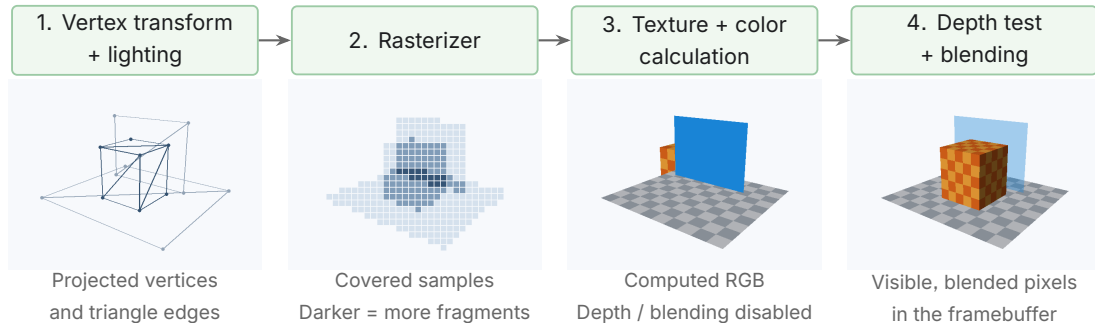
≈ **124 million**

output pixels per second

The opportunity is repeated work across independent data.

A scene becomes pixels, one stage at a time

SAME MESH · SAME CAMERA · SAME DRAW ORDER



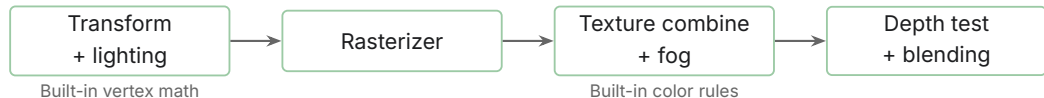
Original software-renderer diagnostics · logical stages; hardware may test depth early

Vertices → fragments → fragment colors → framebuffer pixels.

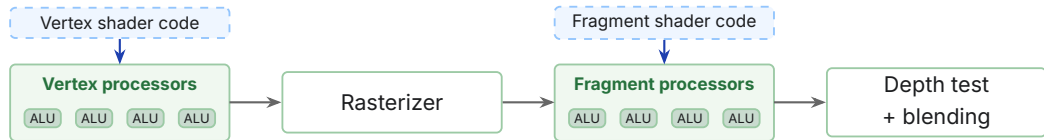
Pipeline reference: NVIDIA · The Cg Tutorial, Ch. 1 · original scene and diagnostic renders

Richer materials needed programmable hardware

EARLY OPENGL · SETTINGS FOR FIXED-FUNCTION HARDWARE



EARLY PROGRAMMABLE GPU · APPLICATION SUPPLIES PROGRAMS



Pipeline order, not a floorplan · texture sampling hardware remains · output goes to the framebuffer

Custom material formulas need both code and processors that can execute it.

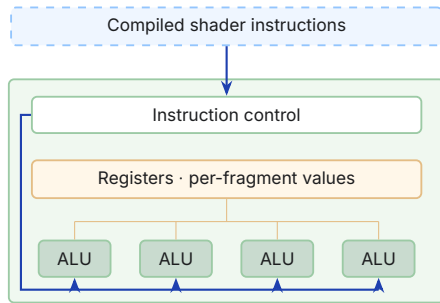
NVIDIA · The Cg Tutorial, Chapter 1 · simplified hardware stages and illustrative unit counts

Shader code becomes instructions for physical processors

SOFTWARE · ONE FRAGMENT'S WATER SHADING

```
normal = ripple_normal(uv, time)
reflected = sample_reflection(view, normal)
weight = reflection_weight(view, normal)
output = mix(water_color, reflected, weight)
```

- The program chooses the calculation.
- Each fragment supplies different inputs.



The same units process many fragments.

Code supplies instructions; registers hold values; ALUs perform the arithmetic.

Do those numbers have to be colors?

THE GRAPHICS INTERPRETATION



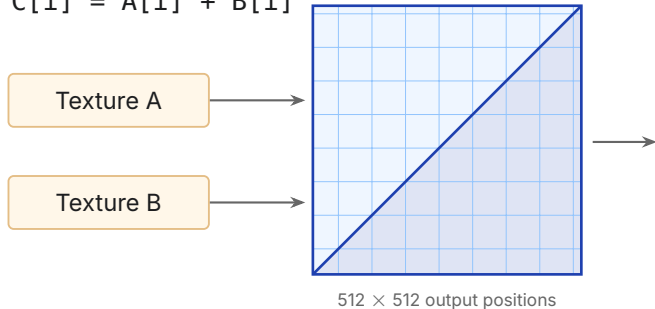
THE COMPUTE INTERPRETATION



GPGPU: General-Purpose Computing on GPUs

To add two arrays, first draw two triangles

$$C[i] = A[i] + B[i]$$



- 1 Set the output image size.
- 2 Cover it with two triangles.
- 3 Add values in a fragment shader.

Interpret the output as an array.

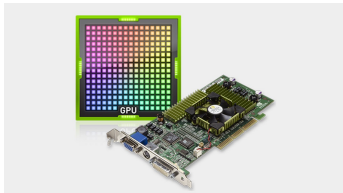
A draw call to the GPU; an array operation to the researcher.

“All this just to do $A + B$ ”



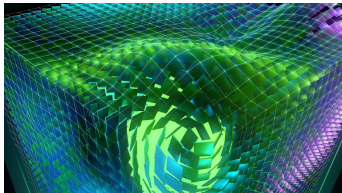
- The entry point still speaks graphics.
- Fragment programs have restricted output locations.
- Sharing data between executions is constrained.

CUDA arrived with a more general GPU



1999

GeForce 256



2006

CUDA architecture

`launch()`

CUDA Toolkit 1.0

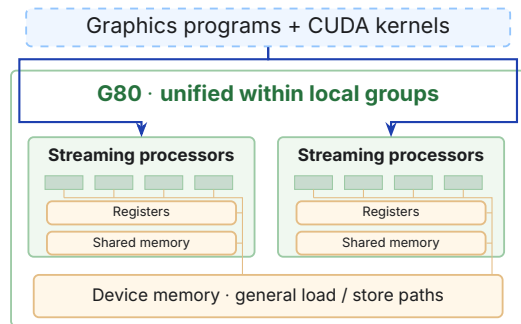
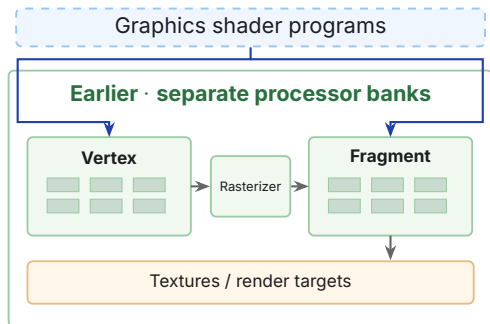
2007

CUDA Toolkit 1.0

A compute programming model arrived with a GPU that could execute it.

Images / 2006: NVIDIA corporate timeline · Toolkit 1.0: NVIDIA guide, June 2007

CUDA used unified processors and new memory paths



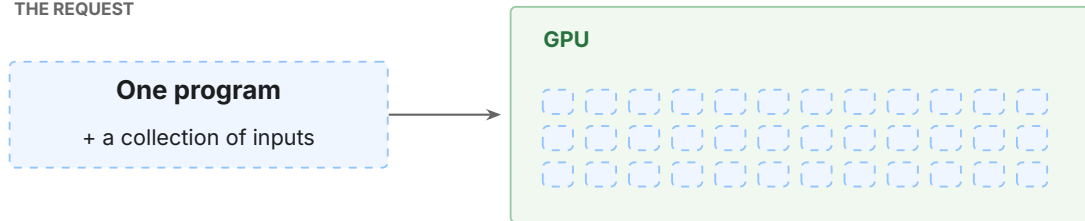
Selected resources · graphics rasterization remains · processor counts are illustrative

CUDA needed executable instructions, direct data access, and local cooperation.

NVIDIA · GeForce 8800 Technical Brief, pp. 12–14, 21–25 · CUDA Guide 1.0, pp. 2–5

“Run this program many times.”

THE REQUEST



The data-parallel structure survives.

Many logical executions

Now look inside the machine that accepts that request.

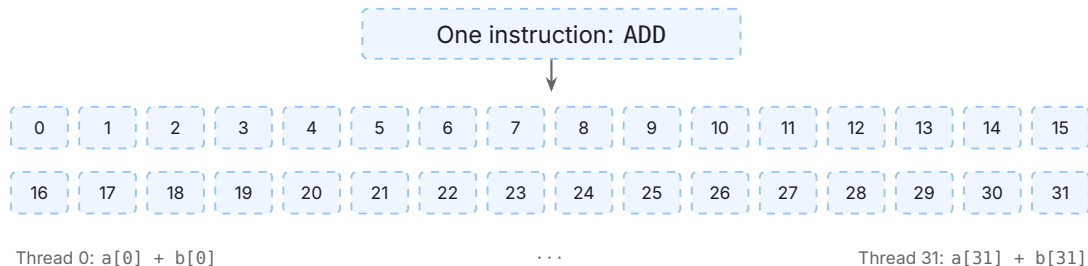
2. From pictures to programs

3. The execution engine

- Warps and SIMT
 - SMs and GPU organization
-

4. Following A + B

Hardware groups 32 logical threads into a warp

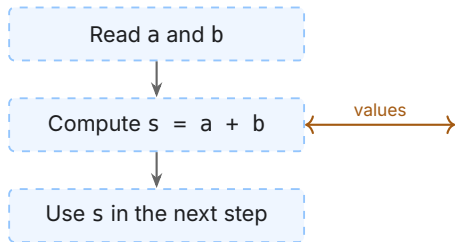


SIMT Single Instruction, Multiple Threads

A warp groups thread execution; execution units do the work.

Each thread needs values that survive an instruction

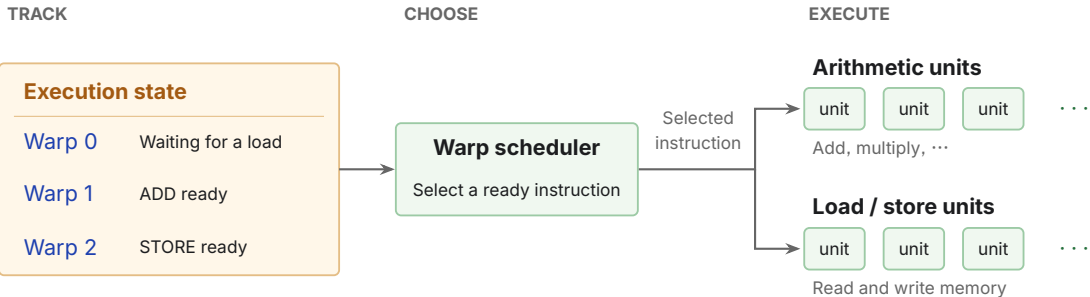
ONE WARP · SAME STEPS, DIFFERENT VALUES



REGISTER FILE			
Thread	a	b	s
0	a[0]	b[0]	s[0]
1	a[1]	b[1]	s[1]
	...		
31	a[31]	b[31]	s[31]

A register file holds working values; each thread has its own registers.

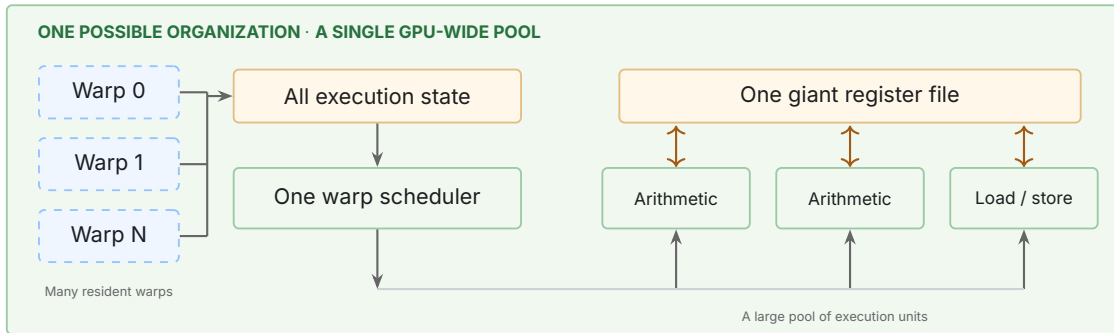
Track progress, choose an instruction, execute it



Required roles; unit counts and connections are illustrative.

Track each warp's state. Issue selected instructions to execution units.

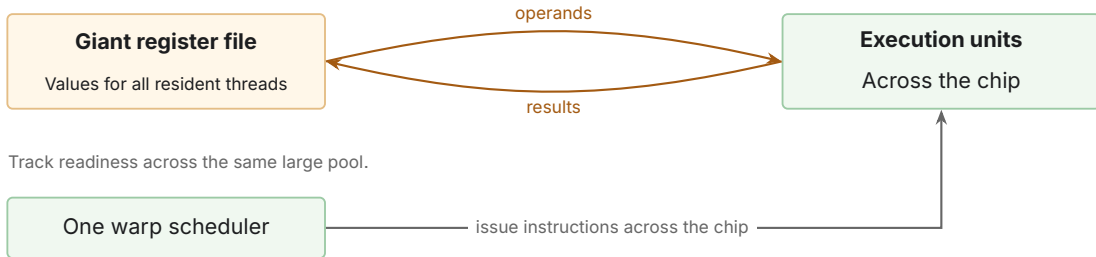
Why not build one giant execution engine?



One place for values. One place to choose work. Many units to execute it.

The values and control signals must travel too

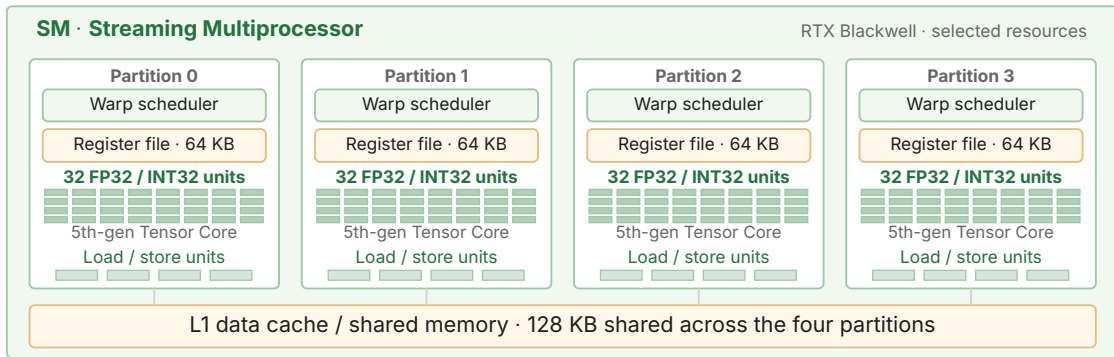
THE SAME CENTRALIZED DESIGN · NOW CONSIDER THE CONNECTIONS



Track readiness across the same large pool.

More units also need more register bandwidth and a larger control network.

An SM keeps execution and its working state close

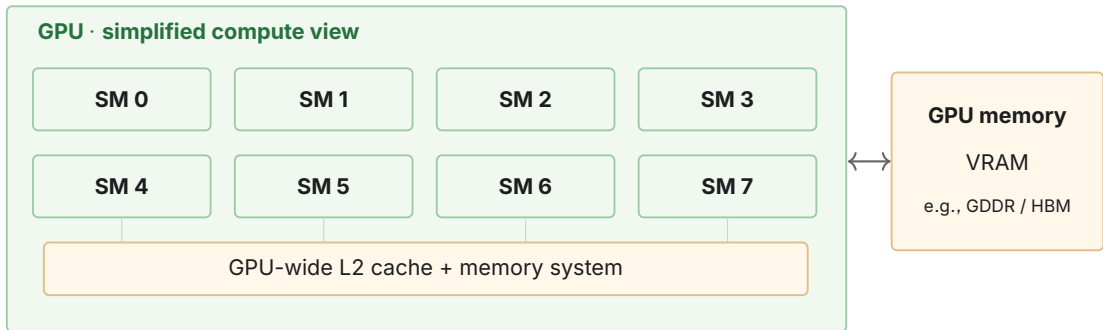


Each scheduler manages many warps using its partition's local resources.

NVIDIA RTX Blackwell whitepaper · Figure 5, pp. 10–12 · simplified and rearranged

The execution engine

Scale the GPU by repeating that execution engine

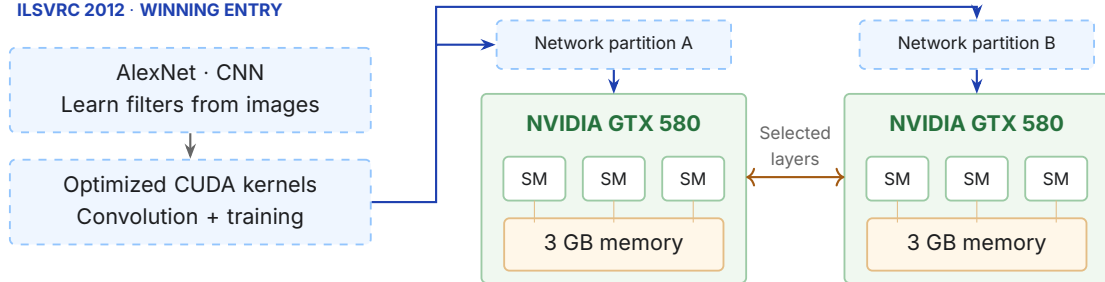


Share control within a warp; keep execution resources local to an SM.

NVIDIA CUDA Programming Guide · GPU Hardware Model · illustrative SM count

2012: AlexNet trained on two NVIDIA GPUs

ILSVRC 2012 · WINNING ENTRY



One network: about 5–6 days to train

Memory limits shaped the split; communication shaped the connections.

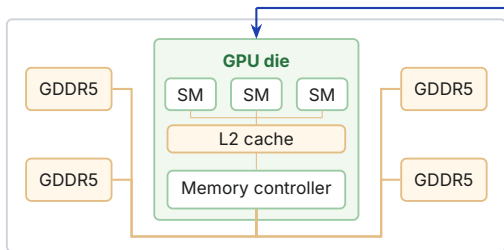
Existing GPU compute made large-scale learning practical; the workload then kept growing.

Krizhevsky, Sutskever & Hinton · 2012 · §§3.2, 5–6 · Original CUDA code · CHM archive · schematic partitions and SM count

More memory traffic changed the GPU package

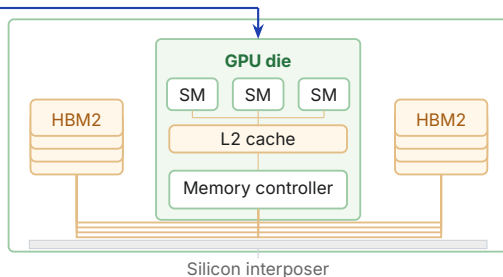
CUDA kernel · load inputs → compute → store outputs

GDDR5 · SEPARATE PACKAGES ON A BOARD



Same load / store model · schematic components and wiring

P100 · 2016 · GPU AND HBM2 IN ONE PACKAGE



HBM widens the physical data path; kernels still need efficient access and reuse.

NVIDIA · Inside Pascal (2016) · Dramatic Improvements in Memory · schematic packaging comparison

Matrix kernels gained a new physical execution path

MATRIX MULTIPLY + ACCUMULATE · $D = A \times B + C$

GEMM kernel · repeated multiply-adds

FMA instructions: $d \leftarrow a \times b + d$

Pascal SM · selected resources

Registers · operands and accumulated values

ALU ALU ALU ALU ALU ALU

CUDA 9 · library or WMMA kernel

Matrix multiply-accumulate instructions

Volta SM · selected resources

Registers · operands and accumulated values

ALU ALU ALU

Ordinary ALUs remain

Tensor
Core

Tensor
Core

Example: FP16 matrix inputs, FP32 accumulation · illustrative unit counts · instruction throughput is not shown

Tensor-aware kernels and instructions use added matrix hardware.

NVIDIA · Programming Tensor Cores in CUDA 9 (2017) · Pascal FMA and Volta Tensor Core paths

Generating text adds a growing state to reuse



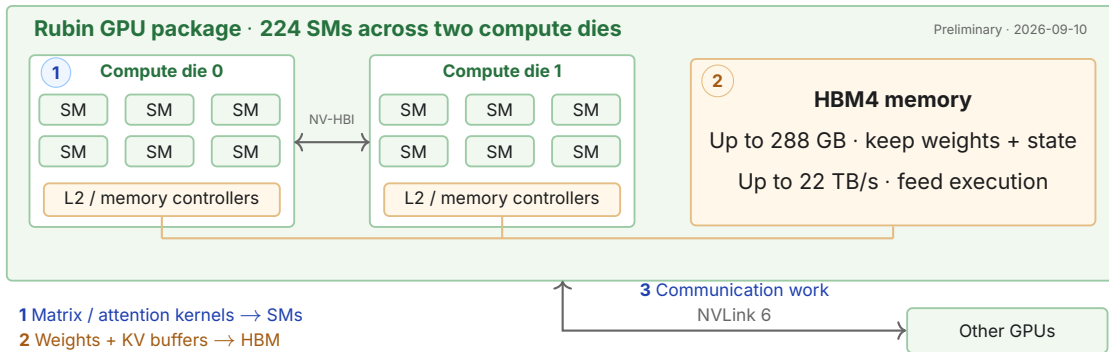
Longer context + more requests → more stored state

Generating tokens → repeated access to weights and state

A model spread across GPUs → communication

The bottleneck depends on the model, batch size, and execution phase.

Rubin scales compute, memory, and communication



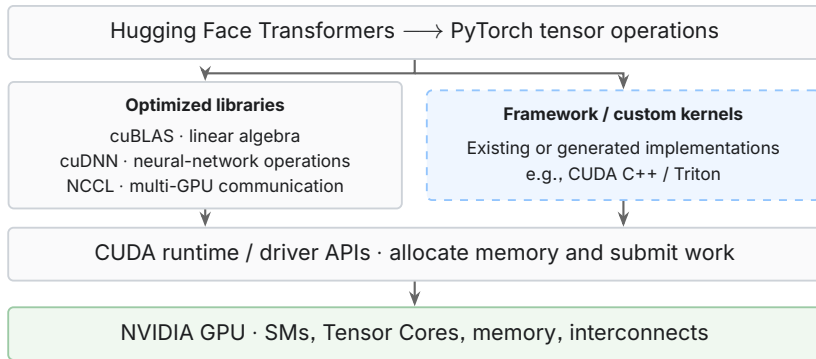
The workload shapes the arithmetic, storage, and connections together.

NVIDIA · Inside Rubin GPU Architecture · July 2026 · SM counts, wiring, and software mappings are schematic

4. Following A + B

- From tensors to threads
- Scheduling and memory
- Results and completion

CUDA connects frameworks to the hardware



Build

Compilers
e.g., nvcc

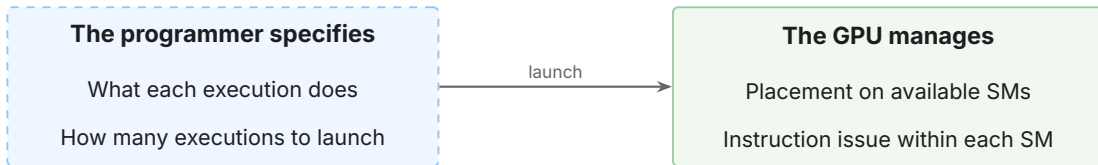
Measure

Nsight Systems
Application timeline
Nsight Compute
Individual kernels

Libraries, kernels, compilers, and tools make hardware capabilities usable.

NVIDIA CUDA platform and libraries · PyTorch / Hugging Face docs · example execution paths

Describe the work; the GPU places it



PyTorch chooses CUDA implementations underneath tensor operations.

A work description is not a physical placement plan.

Start with one line of PyTorch

```
import torch  
A = torch.randn(1024, 1024, device="cuda")  
B = torch.randn(1024, 1024, device="cuda")  
C = A + B
```

A



B



+

=

C



Each output needs only its own two inputs. No neighbor must finish first.

If the input starts in CPU memory, copy it to the GPU

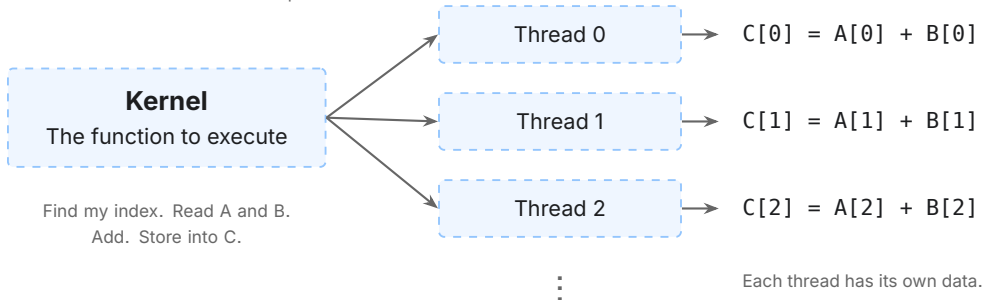


```
A_cpu = torch.randn(1024, 1024, device="cpu")  
A = A_cpu.to("cuda")
```

With `device="cuda"`, our original example already creates GPU tensors.

One program, many logical executions

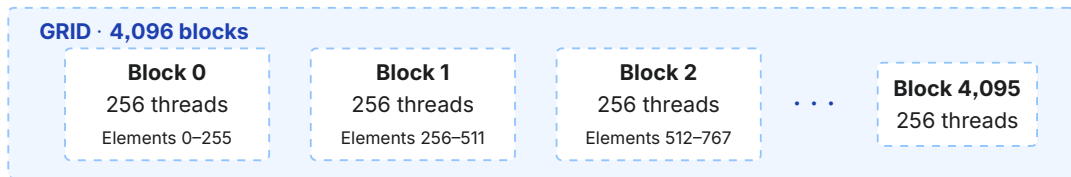
Educational kernel · one element per thread · matrix elements numbered as a flat buffer



A kernel is code. A thread is one logical execution of that code.

How many executions do we launch?

$1024 \times 1024 = \mathbf{1,048,576}$ elements



```
// CUDA C++: A/B/C buffers are ready; device_a/b/c are their GPU addresses.  
add<<<4096, 256>>>(device_a, device_b, device_c, 1024 * 1024);
```

4,096 blocks $\times$ 256 threads = 1,048,576 logical threads.

Every thread can find its own element

```
__global__ void add(  
    const float* a,  
    const float* b,  
    float* c,  
    int element_count  
) {  
    int index = blockIdx.x * blockDim.x  
        + threadIdx.x;  
    if (index < element_count) {  
        c[index] = a[index] + b[index];  
    }  
}
```

Same code, different built-in IDs, different data.

BLOCK 3, THREAD 10

$$\underbrace{3 \times 256}_{\text{block offset}} + \underbrace{10}_{\text{local index}}$$

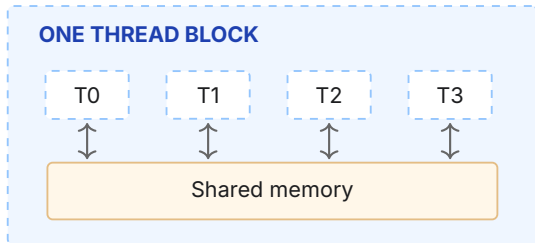
778

C[778] = A[778] + B[778]

The bounds check protects the final block when the size is not divisible by 256.

`__global__`: GPU kernel `.x`: 1D launch IDs

A block is also a cooperation boundary



Share data locally

Threads in a block can exchange data.

Synchronize within the block

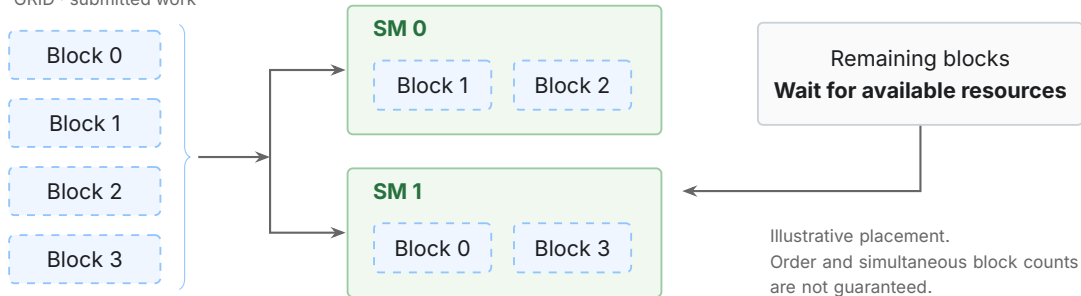
`__syncthreads()` provides a block-wide barrier.

For example, threads can combine intermediate results through shared memory.

This addition needs no shared intermediate values or block barrier.

Many blocks share a finite set of SMs

GRID · submitted work



A block stays on one SM. Multiple blocks let work spread across SMs.

A 256-thread block contains eight warps

BLOCK 0 · 256 threads

Warp 0  0–31

Warp 1  32–63

Warp 2  64–95

Warp 3  96–127

Warp 4  128–159

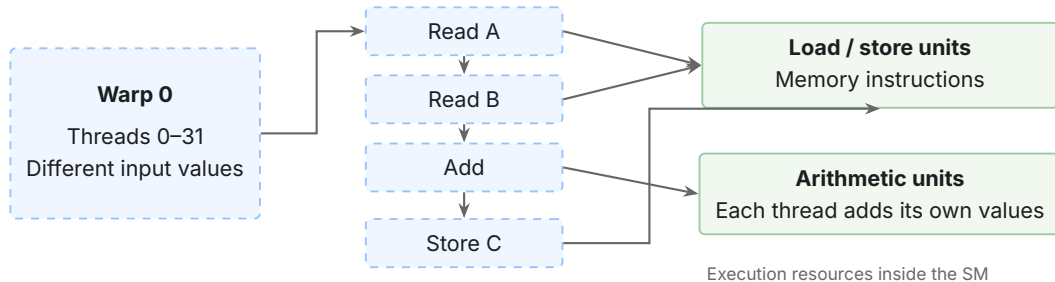
Warp 5  160–191

Warp 6  192–223

Warp 7  224–255

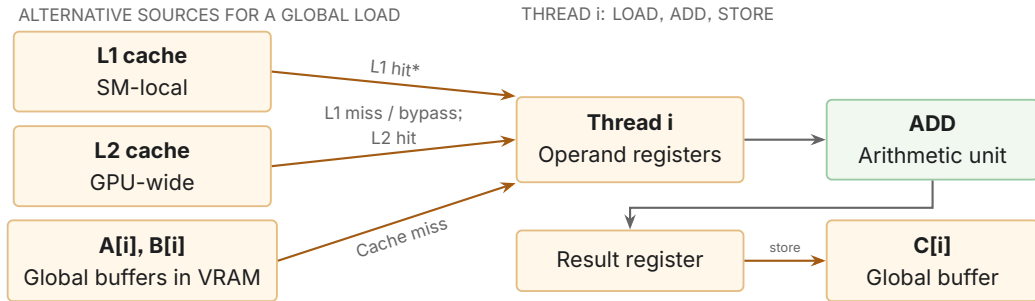
Within a block, consecutive groups of 32 threads form warps.

A warp needs different units for different instructions



The instruction is shared; the data belongs to each thread.

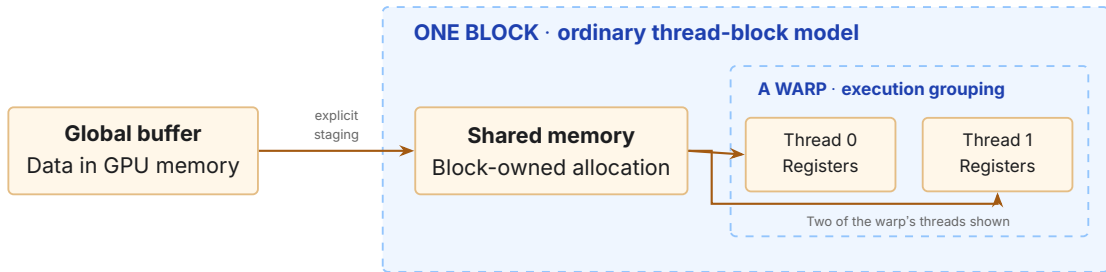
For this addition, operands are loaded into thread registers



*L1 use depends on architecture and policy. Arrows show possible data sources, not physical routes.

The add kernel uses per-thread registers. Shared-memory staging is unnecessary.

Shared memory is explicit storage for block cooperation

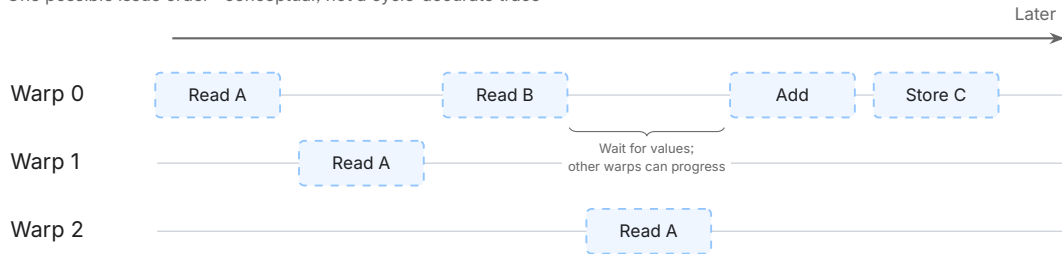


Useful when threads reuse or exchange data. This addition does not take this optional path.

Shared memory is optional staging that the kernel manages for cooperation and reuse.

The scheduler issues the next ready instruction

One possible issue order · conceptual, not a cycle-accurate trace



Registers retain each thread's values between instructions.

A warp does not have to finish its whole program before another runs.

Many resident warps share the same execution units

RTX BLACKWELL · COMPUTE CAPABILITY 12.0

Resident execution state

Up to 48 warps per SM

$48 \times 32 = 1,536$ logical threads

Limited by registers, shared memory, and block size



Physical arithmetic resources

4 partitions $\times$ 32 FP32 / INT32 units

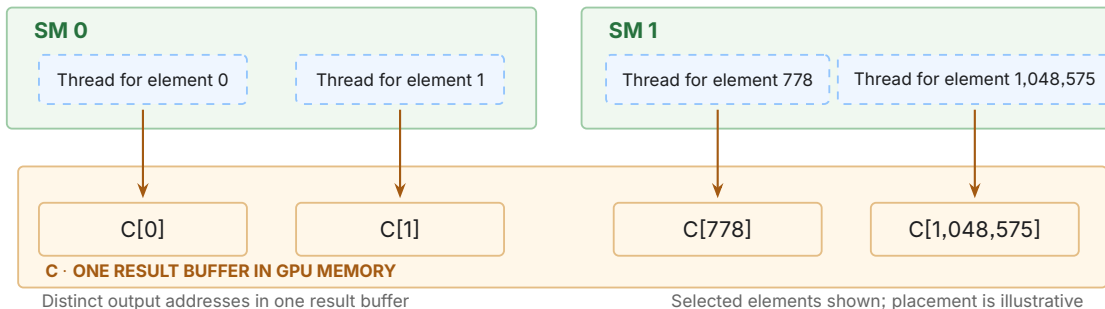
= 128 units per SM

Schedulers issue ready instructions to these units

Resident threads retain their state; they do not each own an arithmetic unit.
Warp width, instruction throughput, and result latency are separate quantities.

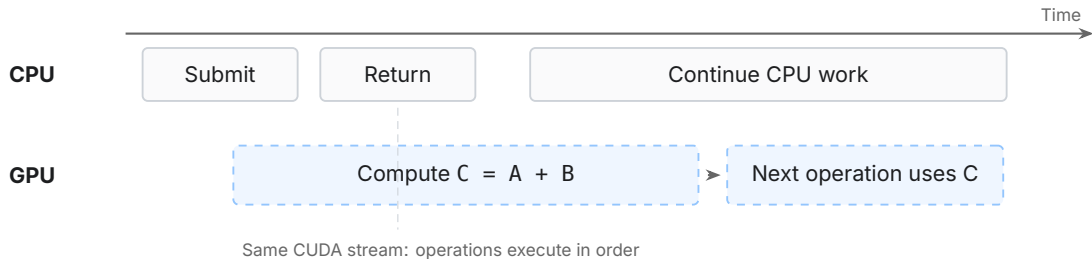
Resident warp capacity is not the arithmetic-unit count divided by 32.

Threads write directly into C



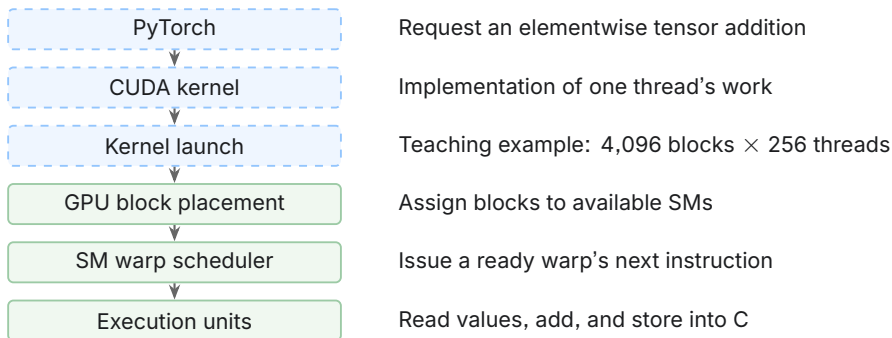
Each thread stores its own result. The CPU does not reassemble the matrix.

Python can return before the GPU finishes



- GPU operations are normally submitted asynchronously.
- Ordinary CPU copies wait as needed; GPU timing must account for completion.

Follow the request all the way down



One tensor operation becomes many scheduled instructions on real hardware.

Keep the work and the machine separate

Kernel	The GPU program to execute
Thread	One logical execution of that program
Block	A group of threads that can cooperate
Grid	All blocks submitted for this kernel launch
Warp	A 32-lane execution group within one block

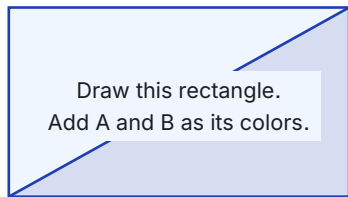
SM	The hardware engine that executes resident warps
-----------	--

Grid $\neq$ GPU. Block $\neq$ SM. Thread $\neq$ a dedicated core.

Following $A + B$

From two triangles to one line

THEN



NOW



$$C = A + B$$

The constant idea: apply the same computation to many inputs.

GPU 101

How a machine for pictures learned to compute $A + B$

Educational scope This introduction uses simplified kernels and conceptual diagrams. Hardware details, memory behavior, and launch configurations vary by architecture and software version; the examples do not specify PyTorch's actual implementation. NVIDIA and Stanford materials are credited for reference and do not imply endorsement. Interpretations and any errors are the presenter's responsibility.

Beyond one GPU: choose a question

Other accelerators

[AMD + ROCm](#) [HIP](#) [TPU](#)
[Torch/XLA](#) [PJRT](#) [StableHLO](#)
[NPU startups](#) [Cerebras](#)

LLMs and the data center

[Training](#) [Serving](#) [Parallelism](#)
[Reliability, power, cooling](#) [NVIDIA networking](#)

Bits, bandwidth, packaging

[Number formats](#) [Mixed precision](#)
[HBM](#) [FlashAttention](#) [TSMC packaging](#)

Linux and deployment

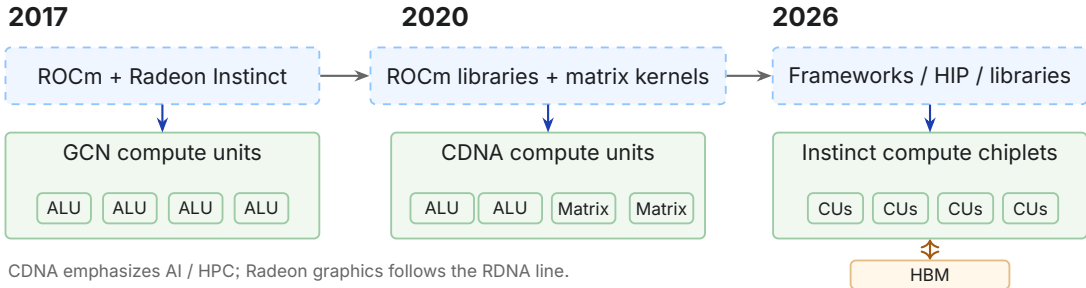
[Torvalds and NVIDIA](#)
[Portable GPU images](#) [Containers and the host driver](#)

The recurring question: what must change around the compute units?

Optional reference topics · [click a topic](#) or use [Appendix index](#) · sources checked September 10, 2026

AMD built a compute platform alongside its GPUs

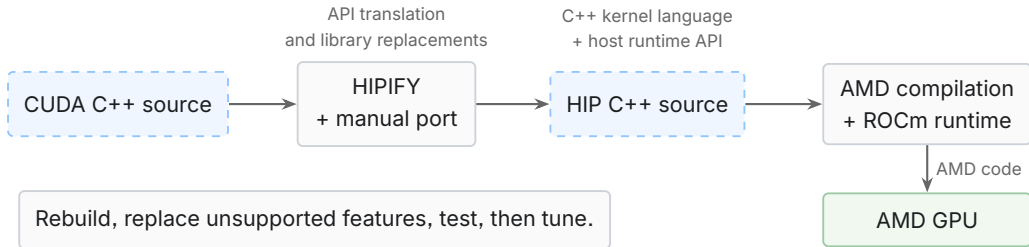
A FEW MILESTONES · HARDWARE AND SOFTWARE TOGETHER



The alternative is a hardware, compiler, library, and tooling stack.

AMD 2017 results · CDNA white paper, pp. 2-7 · CDNA architecture · illustrative units

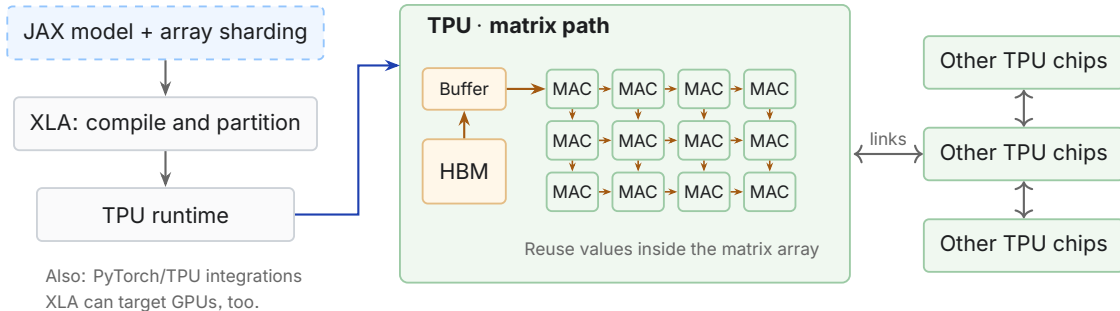
HIP moves source code to a different GPU stack



Source portability does not imply CUDA binary or performance compatibility.

AMD · What is HIP? · C++ runtime API, kernel language, HIPIFY, and porting limits

TPUs start from neural-network dataflow

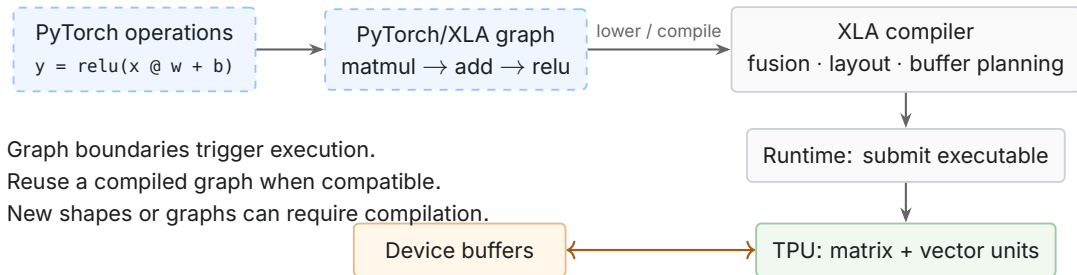


The compiler maps tensor work and communication onto an ML-focused machine.

Google · TPU architecture · JAX stack · TPU, ISCA 2017 · selected datapath, illustrative array and topology

PyTorch/XLA first turns tensor work into a graph

CLASSIC LAZY-TENSOR PATH · BEFORE AND AFTER THE RUNTIME CHANGE



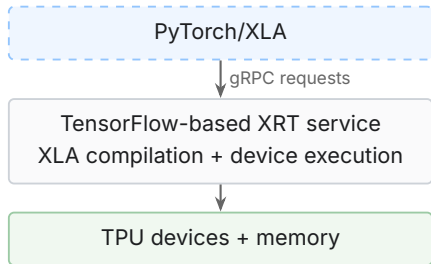
Graph boundaries trigger execution.
Reuse a compiled graph when compatible.
New shapes or graphs can require compilation.

The tensor API looks familiar; the compiler chooses how that work fits the TPU.

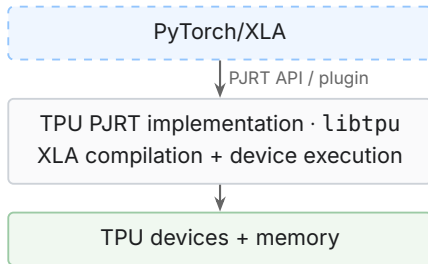
PyTorch/XLA 2.4 · XLA Tensor Deep Dive · classic path, not every PyTorch/TPU execution mode

PJRT replaced the device-runtime connection

BEFORE · XRT



AFTER · PJRT



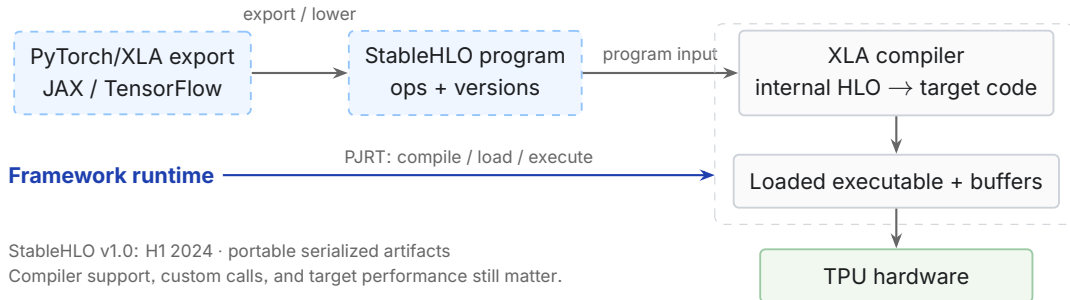
PyTorch/XLA 2.0: default when no XRT configuration · 2.1: stable · JAX also uses PJRT

Less runtime plumbing; a shared API for devices, buffers, compilation, and execution.

PyTorch/XLA · XRT to PJRT, 2023 milestones · OpenXLA · uniform device API · schematic runtime boundaries

StableHLO stabilizes the program, PJRT runs it

BEFORE: HLO / MHLO WERE NOT A VERSIONED PORTABLE CONTRACT

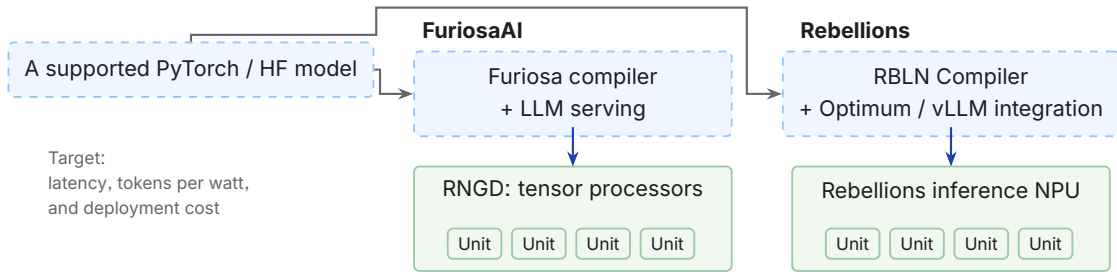


StableHLO v1.0: H1 2024 · portable serialized artifacts
Compiler support, custom calls, and target performance still matter.

StableHLO defines what to compute; PJRT provides the device API used to compile and run it.

StableHLO · compatibility · PyTorch export · PJRT API · selected TPU path; other compilers can consume StableHLO

NPU startups compete on the complete inference service

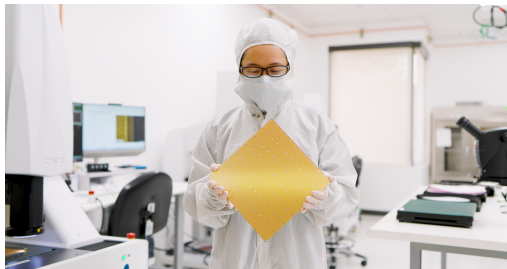


Check the supported operators, shapes, quantization, and serving features for the actual model.

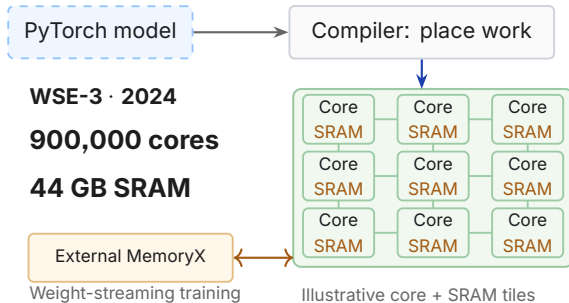
Specialized hardware needs a compiler and serving stack that keep up with models.

Furiosa SDK 2026.2 · Rebellions Inference Platform · examples of inference-focused NPUs; conceptual units

Cerebras makes the wafer itself a compute fabric



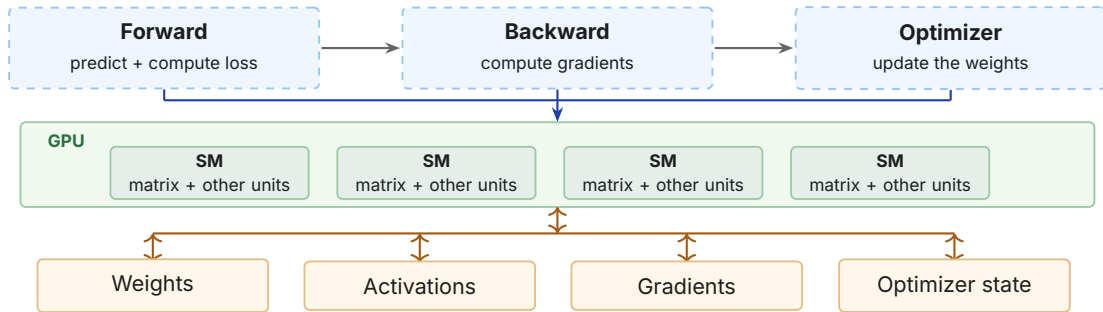
Cerebras wafer-scale processor · hands show physical scale



Move much more compute and memory onto one fabric; the system boundary still matters.

Photo: Cerebras · Cerebras · WSE-3 announcement, March 2024 · PyTorch compiler · Weight streaming

Training keeps much more than the model weights

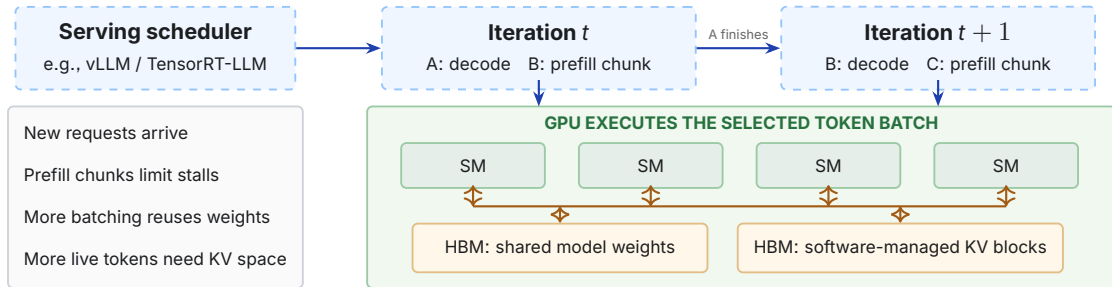


HBM holds tensor state. Sharding spreads state across GPUs; recomputation can save activations.

Fitting the weights is only the start of fitting a training job.

Rajbhandari et al. · ZeRO (2019/2020), §§3–4 · original schematic; physical counts illustrative

LLM serving schedules tokens and memory together

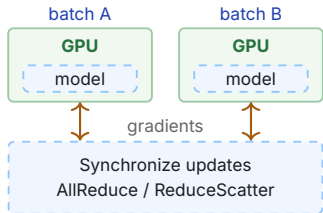


Illustrative schedule. The best mix depends on model, context, hardware, and latency target.

Serving balances time to first token, time between tokens, throughput, and memory.

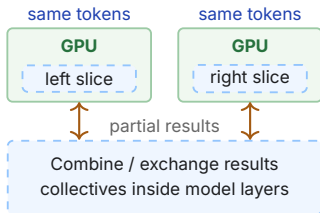
Splitting a model creates different kinds of traffic

Data parallel



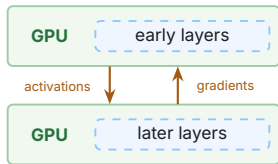
Different samples; same model
state may also be sharded

Tensor parallel



Different pieces of each tensor
frequent communication

Pipeline parallel



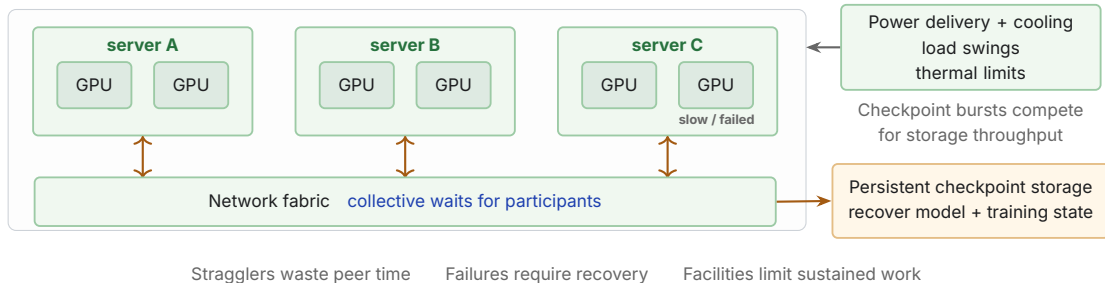
Different layers; micro-batches
pipeline bubbles can leave GPUs idle

Parallelism must fit both GPU memory and the network topology.

Llama Team · The Llama 3 Herd of Models (2024), §3.3.2 · original schematic; these strategies can be combined

At cluster scale, one slow component can stall the job

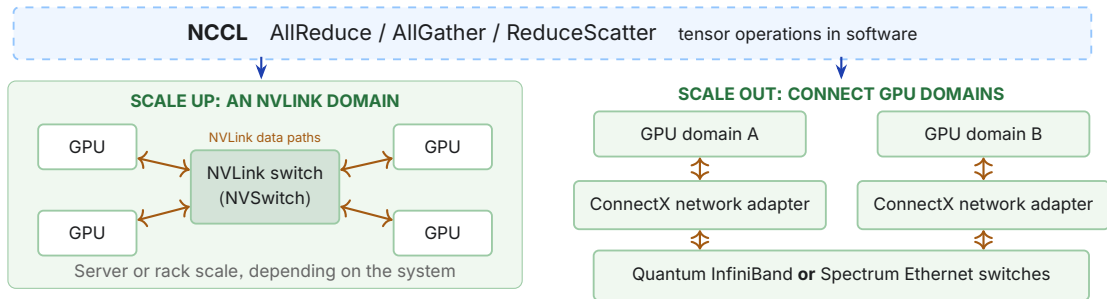
ONE SYNCHRONOUS TRAINING JOB



Useful training time depends on the whole facility, including recovery.

Llama Team · The Llama 3 Herd of Models (2024), §§3.3.1, 3.3.4 · reported production-cluster challenges

NVIDIA also builds the paths between GPUs

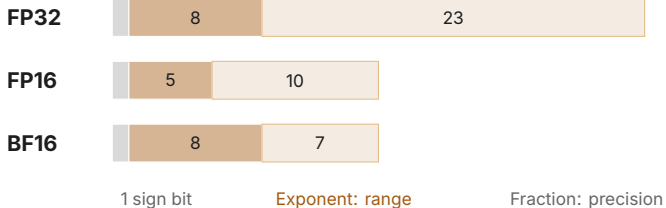


Mellanox joined NVIDIA in 2020. BlueField DPUs offload networking, storage, and infrastructure services.

Fast GPUs need fast communication; NCCL chooses how tensor data moves over hardware.

A number's bits buy range, precision, and cost

STORED FLOATING-POINT VALUES · equal width per bit

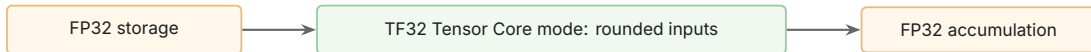


Same 16 bits, different choices

BF16 keeps FP32's exponent range.
FP16 keeps more fraction bits.

More bits still have a role

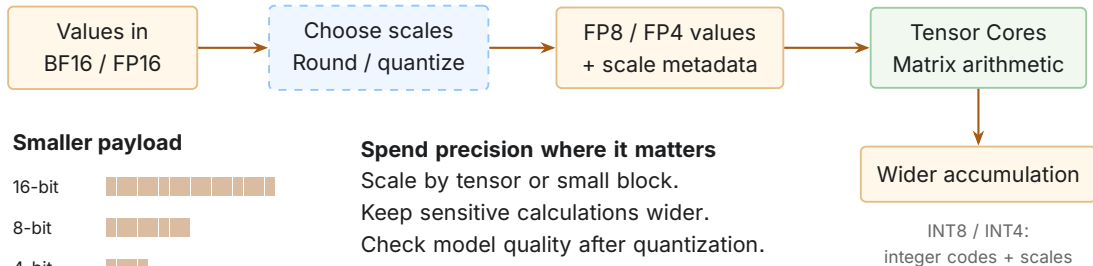
FP64 for demanding numerical work;
FP32 for sensitive calculations.



Precision and range are different. TF32 is a compute mode for FP32 data.

Fewer bits need more numerical engineering

AN EXAMPLE LOW-PRECISION MATRIX PATH · supported hardware and kernels



The trend is mixed precision: smaller operands, selective precision, validated accuracy.

NVIDIA Transformer Engine · FP8 / FP4 and scaling · Micikevicius et al. · Mixed Precision Training (2017)

HBM widens the pipe; arithmetic still needs reuse

OUR STREAMING FP32 ADD · inputs read once, output written once

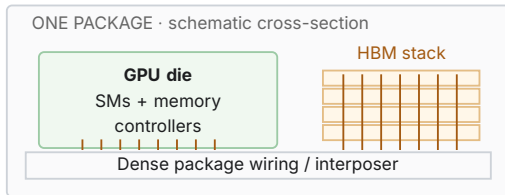


12 bytes / FLOP

An illustrative **4 TB/s** memory system:

$$4 \times 10^{12} / 12 \approx 0.33 \text{ TFLOP/s}$$

Ideal bandwidth ceiling for this add, not GPU peak compute.



Many short parallel connections.

Example: HBM3E has a 1024-bit data interface per stack.

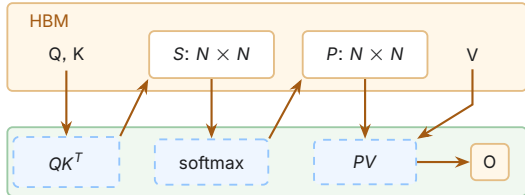
Bandwidth is bytes per second. Reusing each byte can matter as much as moving it faster.

Micron · HBM3E interface · NVIDIA · HBM package topology · A+B arithmetic: illustrative ideal bound

FlashAttention avoids writing the huge score matrix

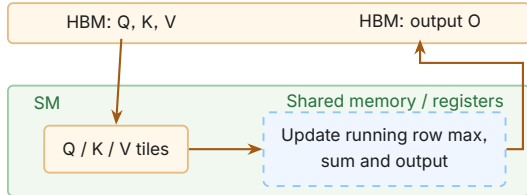
$$O = \text{softmax}(QK^T / \sqrt{d})V \quad \cdot \text{dense attention over } N \text{ tokens}$$

Separate operations



Read / write large intermediates; finally store O in HBM.

FlashAttention · tiled and fused



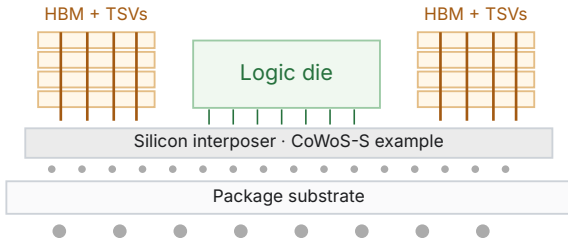
No full $N \times N$ score / probability buffer in HBM.

Same dense attention, less memory traffic. The arithmetic remains quadratic in sequence length.

Dao et al. · FlashAttention (2022), Fig. 1 and Algorithm 1 · schematic data flow; evaluation order and rounding can differ

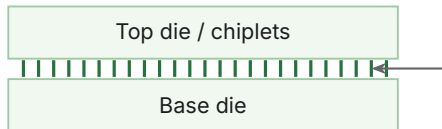
TSMC connects dies sideways and stacks them vertically

CoWoS: integrate logic + HBM in a package



CoWoS-R: RDL. CoWoS-L: local silicon links + RDL.
Schematic; sizes and layer counts are illustrative.

SoIC: dense vertical die-to-die bonds



Fine-pitch bonds · not the HBM DRAM stack

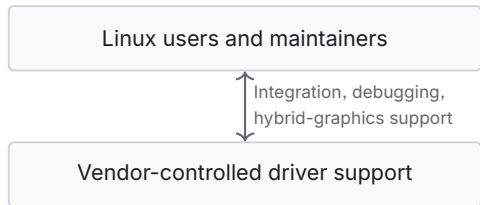
Can be combined with CoWoS.

Constraints: tested dies, bonding yield, heat and package size.

Compute, memory, interconnect, power and cooling must work as one physical assembly.

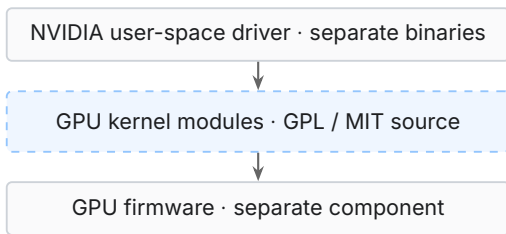
Torvalds wanted Linux to be a supported platform

2012 · THE OPTIMUS QUESTION



The dispute concerned support and cooperation.

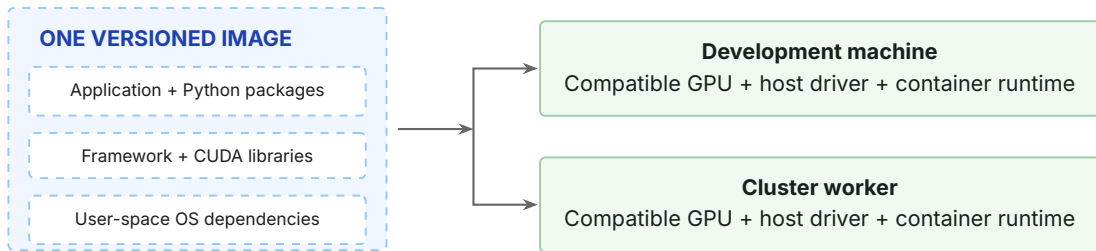
2022 / 2024 · WHICH LAYER OPENED?



R515: source release · R560: wider default adoption

Fast hardware also needs maintainable operating-system integration.

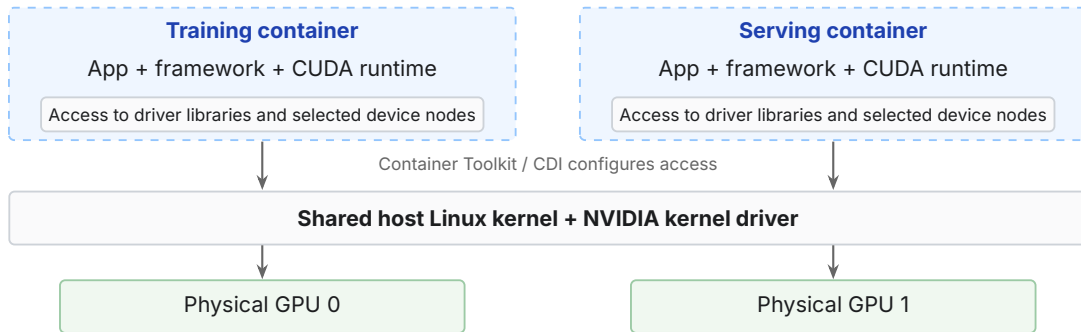
A GPU container carries the application environment



The host need not install each application's CUDA Toolkit and Python environment.

Package the user space once; deploy it on compatible GPU hosts.

The container reaches the GPU through the host driver



Containers isolate application environments; GPU access still crosses a shared driver boundary.